

第 4 组 Web 演示记录（6 场景实测）

- 日期：2026-09-10
- 方式：启动 `examples/miniyu_web.py` (`http://localhost:5000`)，用脚本 `examples/demo_web_scenarios.py` 走与浏览器完全相同的 `/chat` + SSE 链路，逐条提交用户指令，驱动真实 LLM (qwen3.7-flash) function-calling 完成任务。
- 结论：设计书演示场景表 6 个场景全部一次跑通，工具调用链与验收标准一一对应，且经磁盘实体验证（非仅依赖模型自述）。

场景演示结果

场景	用户指令（原文）	实测工具调用链	验收标准核验
1 打 开文 件管 理器	打开文件管理器	<code>run_command(explorer)</code>	✅ 资源管理器窗口打开
2 导 航到 目录	导航到 <code>D:\...\group4_tools_os_skills</code>	<code>run_command(explorer "路径")</code> + <code>list_directory</code>	✅ 列出目标目录内容， 与 GUI 定位一致
3 创 建文 件夹	创建 <code>project</code> 文件夹	<code>create_directory</code>	✅ 磁盘实检 <code>project/</code> 存在
4 文 本编 辑	打开 <code>test.txt</code> 并添加一行	<code>read_text_file</code> → <code>edit_file</code> → <code>read_text_file(复核)</code>	✅ 磁盘实检已追加 <code>demo line from</code> <code>miniyu</code>
5 窗 口切 换	切换到浏览器	<code>list_windows</code> → <code>activate_window(hwnd)</code>	✅ 激活 Microsoft Edge (<code>localhost:5000</code>) 并置 于前台
6 整 理下 载目 录	整理 <code>demo_downloads</code> 按类型分类	<code>list_directory</code> + <code>get_path_metadata</code> ×4 + <code>create_directory</code> ×4 + <code>move_path</code> ×4 (共 14 次调用)	✅ 文件按类型移入 代 码/图片/文本/文档 子文 件夹

场景 6「整理下载目录」完整工具链（复杂任务重点）

```
list_directory(demo_downloads)
→ get_path_metadata(main.py / notes.txt / photo.jpg / report.pdf)    # 摸清类型
→ create_directory(代码)  create_directory(文本)
   create_directory(图片)  create_directory(文档)                    # 按类型建分类夹
→ move_path(main.py→代码) move_path(notes.txt→文本)
   move_path(photo.jpg→图片)  move_path(report.pdf→文档)           # 逐文件归类
→ list_directory(复核)
```

模型自主完成「扫描 → 分类 → 创建文件夹 → 移动」完整闭环，零人工干预。

说明

- 场景 5 选用现存浏览器窗口，验证 `activate_window` 窗口切换能力。
- 场景 6 在隔离的 `demo_downloads/` 测试目录执行（4 个混合类型样本），避免改动真实下载目录。
- 服务运行期间无 `Traceback`，`/status` 全程 200。
- 演示驱动脚本：`examples/demo_web_scenarios.py`（可复现，`python` 直接运行）。

附加：浏览器视觉定位实测（2026-09-11，真实 Chrome 端到端）

场景：让 AI 助手在真实浏览器里“找输入框→输入中文→截图→按编号点击”，验证 `browser_find`（结构化）与 `browser_inspect`（SoM 视觉）两条定位链路。

- 驱动脚本：`examples/browser_som_dom_demo.py`（不依赖 LLM，直接走 Agent 调用的同一条 `ToolRegistry` 工具面，`python` 直接运行）。
- 实测链路的结果：

步骤	动作	实测结果
1	<code>browser_launch(headless)</code> → <code>browser_navigate(bing)</code> → <code>browser_wait("#sb_form_q")</code>	✅ 页面加载到搜索框就绪
2	<code>browser_find(role="textbox")</code> 局部搜索输入框	✅ 命中 <code>ref=eid:sb_form_q</code> （稳定 id ref）
3	<code>browser_type("湖南大学", target=ref,</code> <code>press_enter=True)</code>	✅ 精确输入中文并回车

步骤	动作	实测结果
4	<code>browser_find(text="搜索")</code> 按文字找按钮 → <code>browser_click(target=ref)</code>	✅ 命中并点击（返回 <code>clicked:true</code> ）
5	<code>browser_inspect(max_elements=30)</code> 截交互元素带编号覆盖层图	✅ 落实际 PNG（如 <code>browser_inspect_*.png</code> ），元素含 <code>num/ref/bbox_css/center_css</code> ，DPR=1.75
6	取首元素 <code>num=1→ref=eid:sb_form_q</code> → <code>browser_click(target="som:1")</code>	✅ <code>num→ref→真实鼠标落点</code> （ <code>x=830,y=313</code> ，已按截图 DPR 换算设备像素）
7	结束前 <code>som_session_valid</code>	✅ <code>True</code> （SoM 会话未失效、ref 稳定）

- 结论：结构化优先（find/snapshot→ref 精确操作）与视觉兜底（inspect→SoM 编号→真实鼠标）在真实 Chrome 上全链路自证可用；动作后返回客观 `url_changed / page_changed` 供 Agent 校验。补齐了此前浏览器“只能读、难以稳点”的盲点（记录于 `docs/第4组联调与集成报告.md` §4.3）。

附加：豆包网页端全流程实测 + Token 性能分析（2026-09-11，真实 LLM）

场景：`python examples/miniyu_web.py` 驱动，让 AI 助手“打开浏览器 → 进入豆包聊天页 → 发送『你好』 → 读取回复”。

- 结果：✅ 成功闭环（浏览器可见操作全程前置，未逃逸桌面工具，读取到豆包回复）。
- 过程记录：本次运行产生了唯一一份 `run_log_*.jsonl`（位于 `%LOCALAPPDATA%\Temp\miniyu_artifacts\<session>\`），逐行含时间戳/想法/操作/token。以下是按步还原的 token 曲线：

步	动作	输入(prompt)	输出	total	携带图片数	图片 token	消息数
1	<code>browser_launch</code>	12,895	198	13,093	0	0	2
2	<code>browser_navigate</code>	11,592	94	11,686	1	2,466	5
3	<code>browser_snapshot</code>	14,400	47	14,447	2	4,932	8
4	<code>browser_find(发消息)</code>	16,046	1,245	17,291	2	4,932	10
5	<code>browser_find(textarea)</code>	22,065	146	22,211	3	7,398	13
6	<code>browser_type(div[contenteditable])</code>	25,123	3,292	28,415	4	9,864	16
7	<code>browser_click(发送按钮 952,916)</code>	27,212	227	27,439	5	12,330	19
8	收尾回复	29,284	148	29,432	6	14,796	21

Token 汇总：单任务内容级累计 $\sim 164,014$ (`run_log cum_total_used`)；API 账户口径从 858,780 $\rightarrow$ 584,174，账单 $\sim 274,606$ 。差值约 110k $\approx$ 模型思考/推理 token（深度思考模型在每次工具调用前后产生大量英文复述，公子 API 按 `thinking` 单计，而 `run_log` 的 `completion_tokens` 未含该部分）。

关键结论（数据归因，非猜测）：1. 输入曲线单调上涨 12.9k $\rightarrow$ 29.3k，主因是“截图逐轮累积重发”：步 1 $\rightarrow$ 8 图片数 0 $\rightarrow$ 6，图片 token 0 $\rightarrow$ 14,796，到末轮输入里约 50% 是历史截图。这正是 重复编码已存在的旧图，是当前第一可砍项，而非“单图过贵”。2. 出现了两次无效 `browser_find` 找到 []（发消息占位符/textarea 均非真输入框，输入框是 `contenteditable` div），多烧 2 轮约 3.9 万输入（含重传截图）。改进：snapshot/find 直接暴露 `contenteditable`，并把“发送”语义收敛到点击发送按钮。3. 模型思考冗长：末步 `thought` 全文反复复述“先试 selector... 失败就 click”，输出侧在步骤序列之外烧掉大量 `thinking` token，是账单与日志差 110k 的来源。4. 对比上一轮失败的 9.26 万 token：本轮已修复“工具混用/逃逸”（全程 `browser_`），但绝对成本更高*，说明成本瓶颈已从“工具选择错误”转移到“观察(截图)生命周期 $\times$ 模型思考”，正好验证 `run_log`/观测体系的收益。

下一刀建议（按收益排序）：① 截图只按需触发 + 压缩 + 旧图从 LLM 历史剔除（做 Observation 生命周期）；② 遮蔽 `contenteditable` 定位 + 发送走按钮点击；③ `run_log` 补录 `thinking` token（已加 `reasoning_tokens` 字段），下次即可精确对账账单差。

实施进度（2026-09-11 晚）：① 已落地 —— `_MAX_HISTORY_IMAGES=2` 历史图片裁剪 + `_browser_action_needs_visual` 成功动作不自动截图；③ 已落地 —— `reasoning_tokens` 字段。
② `contenteditable` 定位增强与发送按钮语义作为下一轮。待再跑同款豆包任务验证图片数 6 $\rightarrow$ 0~1、最大单轮 prompt 明显下降。